```
Output:
bool(true)
```

Of course, in the case of İ, a conversion to lower case would also work, but that's purely circumstantial. `mb_convert_case()` doesn't pay attention to locale, and the normal rules just happen to produce identical results for that one character. Case-folding is the tool to use if you want to test equivalence between characters that look identical to human eyes, but be warned that the product of the conversion isn't intended for rendering. Canonical equivalence for machines isn't always going to result in case conversions that look correct to the human eye.

We've spent rather a lot of time looking at what is ostensibly a single function and mainly one weird regional mapping edge case, and we haven't even exhaustively covered everything `mb_convert_case()` does – though the internal workings of the function have been explained, and I'm sure you can figure the rest out for yourself! The point of the last few pages is really to drive home the following point: Unicode is hard. You can't always just throw `mbstring` at a problem and arrive at the correct solution (though multi-byte safety is a very good start). Test. Your. Code. With. Good. Data. Sets.

Edge

After that reasonably deep dive into Unicode's internals, you're probably ready for something a little lighter. The next of the functions are much simpler and only affect the beginning or end portion of an input string.

Table 8-18. *Edge transform functions*

`ltrim()`, `mb_ltrim()` `rtrim()`, `mb_rtrim()`	Strip whitespace from the beginning or end of a string
`trim()`, `mb_strim()`	Strip specified bytes from both ends of a string
`str_pad()`, `mb_str_pad`	Add a specified byte sequence to the beginning or end of a string with a specified length
`mb_strimwidth()`	Truncate string to specified character width